Finite-Shot Quantum Reservoir Computing on Lorenz-96 Against Tuned Classical Baselines

Tushar Pandey
Texas A&M University

October 19, 2026

Abstract

Does a quantum reservoir computer (QRC) forecast a chaotic system better than a classical model when the comparison is fair? We fix four conditions for a fair comparison: every classical baseline is tuned with a search budget comparable to the QRC's, the classical models see the same input window and have the same number of features, the QRC's headline number is read from a finite number of measurement shots, and every result is a multi-seed statistic. Under these conditions we run eight cells on one-step forecasting of a six-variable Lorenz-96 system with a twelve-qubit reservoir: two memory schemes (reset and recurrent), two Hamiltonians (a disordered XXZ chain and an all-to-all Ising model) and two readouts (one measurement basis or two). In all eight cells the QRC loses to a random-feature map of the same width and window, on every one of five fresh data seeds. The best QRC cell ties a degree-2 polynomial of the input window, and an echo-state network of the same width wins every recurrent cell by a wide margin. Measuring a second, non-commuting basis is the largest single gain for the QRC, but it does not close the gap. In one XXZ, single-basis pair per memory scheme, finite shots cost the recurrent QRC far more than the reset one, and a local refinement stage after the Bayesian search, tested in the best QRC cell, improves no model. In free-running forecasts no model's mean valid prediction time reaches one Lyapunov time, and the QRC stays behind the random features and the echo-state network; with one basis it falls to a tie with the linear model (reset memory) or the polynomial (recurrent memory), and with two bases it outlasts the polynomial, which diverges on its own output. All tables are generated from the stored result records.

1 Introduction

Quantum reservoir computing trains only a linear readout on the measured outputs of a fixed quantum system driven by the input signal [1]. Because nothing quantum is trained, it is a candidate for near-term hardware, and a number of studies report that a small quantum reservoir forecasts chaotic or nonlinear time series better than a classical one [2, 3, 4]. Whether such a result says something about the quantum system or about the comparison depends on how the classical side was set up. In earlier work we found that two commonly cited QRC advantages did not survive once the classical baseline received the same tuning and capacity as the quantum model [5], and that a high-dimensional quantum reservoir did not gain from its dimension when matched against classical models of the same width [6]; a related study compared fixed-reservoir and variational quantum models on the Lorenz-63 system [7].

This paper asks the same question on a harder target and under stricter conditions. We call a comparison fair when four conditions hold:

1. every classical baseline is tuned with a budget comparable to the QRC's, by the same search method, on the same tuning seeds;

2. the classical models see the same input window as the QRC, and those built to match it have the same number of features;

3. the QRC's headline number is computed from a finite number of measurement shots, as hardware would report it, not from exact expectation values;

4. every reported number is a statistic over fresh data seeds and reservoir seeds that were never used in tuning, with paired tests corrected for multiple comparisons.

We run eight cells on one-step forecasting of Lorenz-96 with $K = 6$ variables and a $q = 12$ qubit reservoir. The finding is plain. In all eight cells the QRC loses to a random tanh feature map of the same width and window on every data seed (it wins 0 of 25 runs and 0 of 5 data seeds in every cell; the largest Holm-corrected p is 2.3×10^{-4}). The best QRC cell ties a degree-2 polynomial of the input window, and in the recurrent cells an echo-state network (ESN) of the same width wins by 0.156 to 0.188 in NRMSE. Run as free-running forecasters, no model's mean valid prediction time reaches one Lyapunov time (the ESN is longest, 0.80), and the QRC still has a shorter mean VPT than the random features in every cell (significantly in 6 of 8) and than the ESN in every recurrent cell. Section 2 sets out the patterns by which advantage claims are usually made, Section 3 the protocol, Section 4 the results, and Sections 5 to 7 what would have to change for a QRC to win here, the limits of the study, and how to reproduce it.

2 How advantage claims are usually made

We list six patterns that recur in QRC benchmarks. None of them makes a result wrong; each makes it answer a narrower question than an advantage claim needs. We map individual papers to the patterns only where their own text states the relevant fact.

1. Resources counted in different units. A quantum reservoir is often sized by its number of qubits and a classical one by its number of neurons. Ahmed, Tennie and Magri [2] report that on Lorenz-63 their best quantum architecture at 9 to 11 qubits reaches valid prediction times similar to those of classical reservoirs of 500 to 2000 neurons (their Fig. 8). Qubits and neurons are not the same resource for a linear readout: what the readout sees is the number of measured features. We match that number directly.

2. Untuned or lightly tuned baselines. The strength of a baseline depends on which of its knobs are searched. Kobayashi, Fujii and Yamamoto [3] report the best ESN over a grid of five spectral radii and four learning rates (their Appendix D). Nguyen and Soh [4], forecasting the Mackey-Glass series with a driven Jaynes-Cummings reservoir, tune an ESN's spectral radius, leak, input scale and bias over 810 settings on the training loss, while their quantum model receives grid search, Bayesian search and then a further QFI-aware optimisation stage. Ahmed et al. [2] tune hyperparameters with grid search and Bayesian optimisation and find that the classical reservoir outperforms every quantum architecture on Lorenz-63 at the same reservoir size (their Fig. 8), while three quantum architectures outperform it on a ten-variable Lorenz-96 system as the reservoir grows (their Fig. 12). Section 5.2 shows in our own records how one missing knob reverses a result.

3. QRC compared with QRC. Much careful work compares quantum reservoirs with each other: across dynamical regimes [8] or across measurement protocols [9]. That is the right way to design a reservoir, but a gain over another quantum reservoir is not a gain over a classical model, and the two are easy to conflate when the same paper uses advantage language.

4. Capacity metrics in place of forecast error. Memory capacity and expressive capacity measure what a reservoir could represent. Hu et al. [10] give a framework for the expressive capacity of a quantum system under the sampling noise of finite measurement. Capacity is

useful for design, but it does not say whether a tuned classical model forecasts the same task better.

5. Exact expectations in the headline, shots elsewhere. Hardware returns counts. Ahmed et al. [2] report noise-free emulation and leave finite sampling to future work; Kobayashi et al. [3] state their main results under negligible statistical fluctuations and treat the statistical uncertainty of a finite number of measurements in their Appendix C; Nguyen and Soh [4] obtain their margins from expectation-value simulations and analyse finite shots in a supplementary note. Section 4.2 shows that this choice can change the verdict.

6. Few seeds. A single reservoir draw on a single series gives no estimate of the spread between draws, so a gap of a few percent cannot be told from luck. We do not attribute this pattern to specific papers here; we report every comparison over 5×5 evaluation runs.

Where strong classical baselines are included, they often win: in the multivariate gate-based study of Hamhoum et al. [11] the QRC matches classical reservoir computing on Lorenz-63 but a nonlinear vector autoregression and a clustered ESN remain stronger in some settings.

3 Protocol

All runs use the `qrc_bench` pipeline (Section 7). Every constant in this section is read from the pipeline source or from the stored records.

3.1 Task

Lorenz-96 with $K = 6$ variables on a ring, $\dot{x}_k = (x_{k+1} - x_{k-2})x_{k-1} - x_k + F$ with $F = 8$, is integrated by fourth-order Runge-Kutta with step 0.01; one sample is kept every 25 steps, after 1000 discarded samples, and each variable is standardised. A series has 2000 samples in the window cells and 3000 in the recurrent cells. The target is the next sample of all six variables. The score is the one-step-ahead NRMSE on the last 15 percent of the series, averaged over the six variables, each normalised by its standard deviation on the test slice, so that NRMSE = 1 is the mean predictor. The readout is fit once on everything before the test slice, after a washout of 50 samples (window) or 400 samples (recurrent).

3.2 Quantum reservoir

The reservoir has $n_{\text{in}} = 6$ input qubits, one per variable, and $n_{\text{mem}} = 6$ memory qubits, $q = 12$. Each input x is angle-encoded as $R_Y(\text{clip}(s\,x, -\pi, \pi))|0\rangle$ on its input qubit, with the encoding scale s tuned in $[0.1, \pi]$. The system then evolves for a time τ, tuned in $[0.1, 10]$, under one of two Hamiltonians, both all-to-all:

$$H_{\text{XXZ}} = \sum_{i<j} (X_iX_j + Y_iY_j + \Delta\, Z_iZ_j) + \sum_i h_iZ_i, \quad h_i \sim \mathcal{U}(-W, W), \tag{1}$$

$$H_{\text{Ising}} = \sum_{i<j} J_{ij}\, X_iX_j + v \sum_i Z_i, \quad J_{ij} \sim \mathcal{U}(0, 1), \tag{2}$$

with $\Delta \in [-2, 2]$, $W \in [0.1, 10]$ and $v \in [0.1, 10]$ tuned. The random fields or couplings are drawn from the reservoir seed.

Memory. In the *window* cells the memory is reset: at every step the memory qubits start in $|0\rangle^{\otimes 6}$ and the last two inputs are driven in order, the input qubits being traced out between drives. The QRC is then a fixed feature map of a two-step input window. In the *recurrent* cells the memory state is carried from step to step, one drive per step, and a feedback term shifts

every input angle by $k_{\mathrm{fb}} \tanh(\bar{m}_{t-1})$, where $\bar{m}_{t-1}$ is the mean measured $\langle Z \rangle$ of the previous step and $k_{\mathrm{fb}} \in [0, 8]$ is tuned.

Readout. The Z/ZZ readout measures all twelve qubits in the computational basis and gives $\langle Z_i \rangle$ and $\langle Z_i Z_j \rangle$, $12 + 66 = 78$ features. The Z/ZZ+X/XX readout measures the same block a second time in the X basis, giving 156 features. The feedback signal and the memory state stay on the Z basis.

Shots. Every QRC feature in the eight cells is estimated from $1024\,q = 12288$ shots per step. With two bases the budget is split equally, 6144 per basis, so the second basis costs no extra shots. We use the restarting protocol [9]: each step's measurement is a fresh preparation of the same circuit, so the memory state carried to the next step is exact and is never collapsed, while the features and the feedback signal are both read from the sampled counts. The shot noise realisation is fixed by the reservoir seed. The exact-expectation runs of Section 4.2 read exact values with the same protocol block, but were made with an earlier pipeline commit (`4538057` against `a902b04`).

3.3 Classical models

Random features. $\tanh(Az_t + b)$, where z_t stacks the same two-step window the QRC sees ($n = 12$ values), $A_{ij} \sim \mathcal{N}(0, \sigma^2/n)$ and $b_i \sim \mathcal{U}(-\beta, \beta)$, with $\sigma \in [10^{-3}, 10]$ and $\beta \in [0, 10]$ tuned. Its width equals the QRC width in every cell. This is the same-window, same-width control for the QRC.

Poly2 and linear. The complete degree-2 polynomial of the window (12 linear terms and 78 products, width 90) and the window itself (width 12). Only the ridge penalty is tuned. Both are deterministic: they have no reservoir seed.

Echo-state network. In the recurrent cells, a leaky ESN of the same width as the QRC, $r_t = (1-a)\, r_{t-1} + a \tanh(W r_{t-1} + W_{\mathrm{in}} x_t + b)$, with spectral radius in $[0.05, 1.5]$, leak $a \in [0.01, 1]$, input scale in $[0.01, 5]$, density in $[0.02, 1]$ and bias scale in $[0, 10]$ tuned. It sees the whole history, as the recurrent QRC does; the window models remain as references at a two-step window. (As tuned, the ESN's memory is short; see Section 4.)

3.4 Tuning

Every model is tuned by Optuna's TPE sampler [12] with the same sampler seed. The ridge penalty α is tuned in $[10^{-6}, 10^{2}]$ (log scale) for every model, on standardised features, and counts as a knob. The budget is matched by search density rather than trial count: a model with d knobs gets at least $25d$ trials and continues until 50 trials pass without a new best, with a cap of 300. Every model in every cell stopped on the patience rule, after 121 to 197 trials for the QRC, 97 to 239 for the random features and 189 to 279 for the ESN. The objective is the validation NRMSE averaged over three rolling-origin folds (each 10 percent of the series, all before the test slice) and over three tuning data seeds $\times$ three tuning reservoir seeds. The best setting of each model is then frozen and evaluated once on five fresh data seeds $\times$ five fresh reservoir seeds, which share no seed with tuning.

3.5 Statistics

For each QRC-versus-classical pair we form the paired difference on each of the 25 evaluation runs and on each of the 5 data-seed clusters (the mean over that seed's five reservoir seeds). We report how many runs and data seeds the QRC wins (a difference within 5×10^{-4} counts

as a tie), a cluster bootstrap 95 percent interval (20000 resamples), and a paired t-test on the cluster means, Holm-corrected over the comparisons in the cell (three in a window cell, four in a recurrent cell). The Holm family is the cell, not the grid: the eight cells hold 28 tests in all, and no correction across cells is applied. The random-feature verdict survives any such correction: the largest uncorrected cluster-level p against the random features, times 28, is 3.2×10^{-3}. The weaker Poly2 results (Holm $p = 0.011$ and 0.023 in the recurrent two-basis cells) may not. For Poly2 and linear the five reservoir seeds of a data seed share one classical prediction, so the run-level tests are not independent; we flag them and use the cluster-level test throughout.

4 Results

4.1 The eight cells

Table 1 gives the test NRMSE of every model in every cell and Tables 2 and 3 the paired statistics; Fig. 1 shows the same numbers.

Table 1: Test NRMSE (mean ± s.d. over 25 evaluation runs, one-step ahead, lower is better) in the eight cells. QRC features are read from 12288 shots per step. Width is the QRC width, which the random features (RF) and the ESN match; readout "+X/XX" is Z/ZZ plus the X basis. Poly2 has width 90 and linear 12. Bold: best model in the cell. Poly2 and linear depend only on the data, so they repeat within a memory kind; the window and recurrent kinds use series of different length.

Kind	Res.	Readout	Width	QRC	RF	Poly2	Linear	ESN
window	XXZ	Z/ZZ	78	0.651 ± 0.018	0.402 ± 0.013	$\mathbf{0.373} \pm 0.013$	0.712 ± 0.010	–
window	XXZ	+X/XX	156	0.407 ± 0.018	$\mathbf{0.324} \pm 0.016$	0.373 ± 0.013	0.712 ± 0.010	–
window	Ising	Z/ZZ	78	0.572 ± 0.013	0.402 ± 0.013	$\mathbf{0.373} \pm 0.013$	0.712 ± 0.010	–
window	Ising	+X/XX	156	0.369 ± 0.012	$\mathbf{0.324} \pm 0.016$	0.373 ± 0.013	0.712 ± 0.010	–
recurrent	XXZ	Z/ZZ	78	0.523 ± 0.018	0.416 ± 0.016	0.392 ± 0.018	0.716 ± 0.010	$\mathbf{0.340} \pm 0.012$
recurrent	XXZ	+X/XX	156	0.421 ± 0.012	0.332 ± 0.017	0.392 ± 0.018	0.716 ± 0.010	$\mathbf{0.259} \pm 0.017$
recurrent	Ising	Z/ZZ	78	0.528 ± 0.012	0.416 ± 0.016	0.392 ± 0.018	0.716 ± 0.010	$\mathbf{0.340} \pm 0.012$
recurrent	Ising	+X/XX	156	0.414 ± 0.009	0.332 ± 0.017	0.392 ± 0.018	0.716 ± 0.010	$\mathbf{0.259} \pm 0.017$

Table 2: QRC wins against each classical model, per cell: evaluation runs (of 25) and, in parentheses, data seeds (of 5, by cluster mean) on which the QRC has the lower test NRMSE. †: the classical model has no reservoir seed, so run-level counts are not independent.

Kind	Res.	Readout	vs Random features	vs Poly2	vs Linear	vs ESN
window	XXZ	Z/ZZ	0/25 (0/5)	0/25 (0/5)†	25/25 (5/5)†	–
window	XXZ	+X/XX	0/25 (0/5)	0/25 (0/5)†	25/25 (5/5)†	–
window	Ising	Z/ZZ	0/25 (0/5)	0/25 (0/5)†	25/25 (5/5)†	–
window	Ising	+X/XX	0/25 (0/5)	14/25 (4/5)†	25/25 (5/5)†	–
recurrent	XXZ	Z/ZZ	0/25 (0/5)	0/25 (0/5)†	25/25 (5/5)†	0/25 (0/5)
recurrent	XXZ	+X/XX	0/25 (0/5)	0/25 (0/5)†	25/25 (5/5)†	0/25 (0/5)
recurrent	Ising	Z/ZZ	0/25 (0/5)	0/25 (0/5)†	25/25 (5/5)†	0/25 (0/5)
recurrent	Ising	+X/XX	0/25 (0/5)	1/25 (0/5)†	25/25 (5/5)†	0/25 (0/5)

Random features beat the QRC everywhere. In every cell the width-matched random-feature map has the lower test NRMSE on all 25 runs and all 5 data seeds, by 0.045 to 0.249, with Holm-corrected $p \leq 2.3 \times 10^{-4}$. The random features see exactly the same two-step window

Table 3: Holm-corrected p of the QRC against each classical model, per cell: paired t-test on the five data-seed means, Holm-corrected within the cell.

Kind	Res.	Readout	vs Random features	vs Poly2	vs Linear	vs ESN
window	XXZ	Z/ZZ	2.7×10^{-6}	2.7×10^{-6}	1.3×10^{-4}	–
window	XXZ	+X/XX	1.3×10^{-5}	6.7×10^{-5}	1.1×10^{-7}	–
window	Ising	Z/ZZ	1.3×10^{-5}	1.3×10^{-5}	3.1×10^{-5}	–
window	Ising	+X/XX	2.3×10^{-4}	0.157	6.5×10^{-8}	–
recurrent	XXZ	Z/ZZ	7.4×10^{-5}	7.4×10^{-5}	1.2×10^{-6}	1.9×10^{-6}
recurrent	XXZ	+X/XX	1.1×10^{-4}	0.011	2.3×10^{-7}	2.2×10^{-5}
recurrent	Ising	Z/ZZ	5.1×10^{-5}	5.1×10^{-5}	2.3×10^{-7}	2.3×10^{-6}
recurrent	Ising	+X/XX	1.4×10^{-4}	0.023	1.2×10^{-7}	2.2×10^{-5}

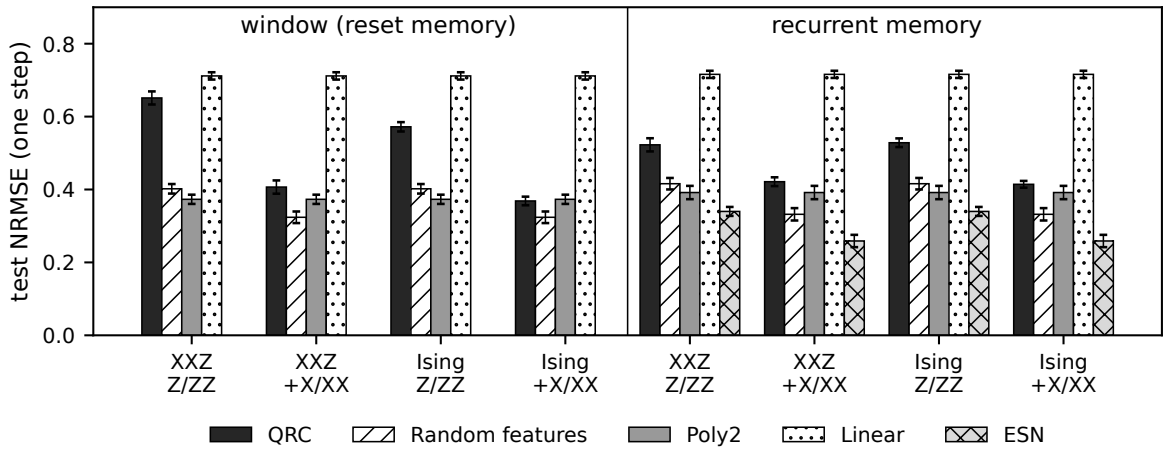


Figure 1: Test NRMSE per model in the eight cells (mean ± s.d. over 25 runs). The random features (hatched) are below the QRC (black) in every cell; the ESN is lowest in every recurrent cell.

and have exactly the same number of features as the QRC, so neither input nor readout width explains the gap.

The second basis is the QRC's largest gain, and it does not close the gap. Table 4 compares each Z/ZZ cell with its Z/ZZ+X/XX partner. Adding the X basis lowers the QRC's NRMSE by 0.244 (window, XXZ), 0.203 (window, Ising), 0.101 (recurrent, XXZ) and 0.114 (recurrent, Ising), although each basis now gets half the shots. The largest of these is the largest change of the QRC between any two cells that differ in one factor. The width-matched classical models gain too, from the doubled width alone: the random features by 0.078 (window) and 0.084 (recurrent), the ESN by 0.081. In the recurrent cells the random features' gain is 74 to 83 percent of the QRC's, so the gap shrinks only by 0.02 to 0.03 (recurrent Ising: 0.112 to 0.082). In the window cells the QRC gains 2.6 to 3.1 times as much as the random features, and the gap to them shrinks from 0.249 (XXZ) and 0.170 (Ising) to 0.083 and 0.045 (last two columns of Table 4), but the QRC still loses on every seed.

Table 4: Effect of the second measurement basis. Columns 3 and 4: QRC test NRMSE with one and two bases (width 78 and 156, same total shots). Columns 5 to 7: change in mean test NRMSE from Z/ZZ to Z/ZZ+X/XX for the QRC and for the width-matched random features (RF) and ESN, whose width doubles with the QRC's. Columns 8 and 9: QRC minus random-feature NRMSE.

		QRC NRMSE		change Z/ZZ → +X/XX			QRC − RF	
Kind	Res.	Z/ZZ	+X/XX	QRC	RF	ESN	Z/ZZ	+X/XX
window	XXZ	0.651	0.407	−0.244	−0.078	–	0.249	0.083
window	Ising	0.572	0.369	−0.203	−0.078	–	0.170	0.045
recurrent	XXZ	0.523	0.421	−0.101	−0.084	−0.081	0.107	0.089
recurrent	Ising	0.528	0.414	−0.114	−0.084	−0.081	0.112	0.082

The best QRC ties Poly2. The best QRC cell (window, Ising, Z/ZZ+X/XX) reaches 0.369 against Poly2's 0.373. It wins 14 of 25 runs, loses 10 and ties 1, and wins 4 of 5 data seeds, but the cluster interval of the difference, $[-0.0090, 0.0001]$, contains zero and the Holm p is 0.157: a tie. The random features in the same cell reach 0.324. In every other cell Poly2 beats the QRC. Every QRC beats the linear model on every run.

The ESN wins the recurrent kind by a wide margin. In the four recurrent cells the ESN of the same width is ahead of the recurrent QRC on every run and every data seed, by 0.156 to 0.188 in NRMSE (Tables 2 and 3). The ESN at width 156 (0.259) is the best model in the whole grid. Its history is long only in name: as tuned, the spectral radius is about 0.05 and the leak 0.84 to 0.88, so the state keeps a weight of only 0.12 to 0.16 on its previous value plus a weak recurrent term, and the past fades within a few steps. The recurrent QRC is not better than the reset QRC with two bases either.

4.2 Shot noise

Table 5 and Fig. 2 compare the XXZ, Z/ZZ cells with runs that read exact expectation values under the same protocol block but an earlier pipeline commit (`4538057` against `a902b04`); in both conditions the QRC is tuned under the condition it is evaluated in. The reset QRC loses 0.019 in NRMSE to the shots (0.632 → 0.651), the recurrent QRC 0.110 (0.412 → 0.523). The difference is consistent with the role of feedback: in the recurrent cells the feedback signal is read from the sampled counts, so measurement noise enters the drive and is carried forward in the memory state, while in the reset cells it touches only the readout. We have not run the ablation

that separates the two (sampled readout with exact feedback), so this is the likely mechanism, not a measured one.

The shots change the verdict, not only the number. With exact expectations the recurrent QRC *ties* the random features: it wins 17 of 25 runs and 4 of 5 data seeds, Holm $p = 0.434$. With 12288 shots it loses to them on every run. A study that reported exact values would have found parity with a tuned, width-matched classical model where hardware would find a loss.

The classical models do not use shots. In the recurrent pair their rows are identical. In the window pair Poly2 and linear are identical; the random features differ (0.406 against 0.402) because the exact run was made with an earlier pipeline version that searched a narrower random-feature range (scale ≥ 0.05, bias scale ≤ 3).

Table 5: Exact expectation values against 12288 shots per step, XXZ reservoir, Z/ZZ readout. Mean test NRMSE over 25 runs; "change" is shots minus exact. In the window pair the random features of the exact run were tuned over a narrower range (see text).

	window, XXZ, Z/ZZ			recurrent, XXZ, Z/ZZ		
Model	exact	12288 shots	change	exact	12288 shots	change
QRC	0.632	0.651	+0.019	0.412	0.523	+0.110
Random features	0.406	0.402	−0.004	0.416	0.416	0
Poly2	0.373	0.373	0	0.392	0.392	0
Linear	0.712	0.712	0	0.716	0.716	0
ESN	–	–	–	0.340	0.340	0

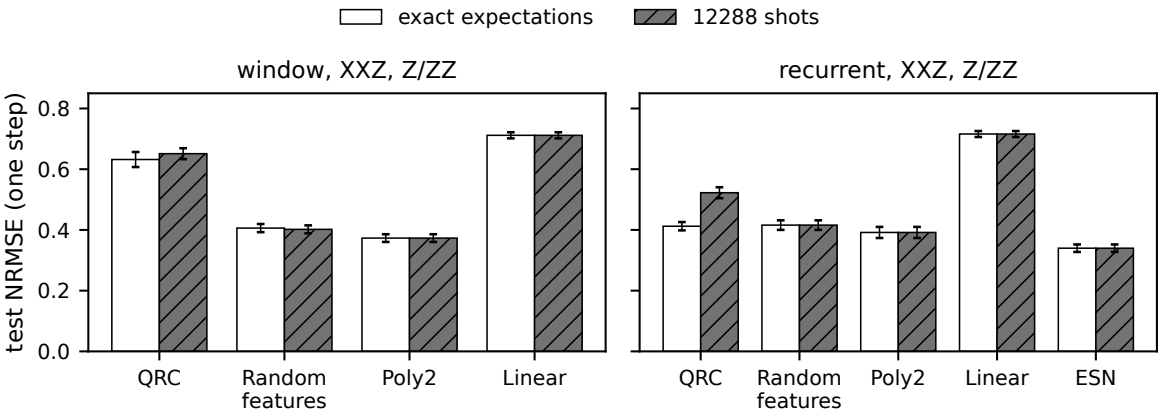


Figure 2: Exact expectation values (white) against 12288 shots per step (hatched), XXZ reservoir, Z/ZZ readout (mean ± s.d. over 25 runs). Only the QRC changes, and the recurrent QRC changes most.

4.3 Is the search finished? A local refinement stage

A tuned comparison is only as good as its search. To test whether TPE had stopped short, we repeated the best QRC cell (window, Ising, Z/ZZ+X/XX) with a refinement stage: after TPE stops, SPSA runs from the best TPE trial in the unit cube of each model's search space, with half the model's floor budget, rounded down to an even number, in further evaluations, for every model. The TPE stage of this run reproduced the original run trial for trial, so the refinement starts from the same point.

The refinement improved no model: the gain is 0 for all four (Table 6, Fig. 3). The landscapes near the optimum are flat for the classical models: 70 percent of the random features' TPE trials,

99 percent of Poly2's and 100 percent of the linear model's lie within 0.02 of their best validation NRMSE. The QRC's landscape is rougher, with 37 percent of its trials within 0.02, but a local search from its best point finds nothing either. Since the frozen settings did not change, the test results of this run are identical to those of the cell in Table 1.

Table 6: SPSA refinement after TPE, window, Ising, Z/ZZ+X/XX. Knobs include the ridge penalty. Validation NRMSE is the tuning objective. "within 0.02": fraction of TPE trials whose validation NRMSE is within 0.02 of the best TPE trial.

Model	knobs	TPE trials	SPSA evals	best TPE val.	best SPSA val.	gain	within 0.02
QRC	4	197	50	0.3845	0.3852	0.0	37%
Random features	3	239	36	0.3570	0.3586	0.0	70%
Poly2	1	81	12	0.4069	0.4070	0.0	99%
Linear	1	111	12	0.7246	0.7246	0.0	100%

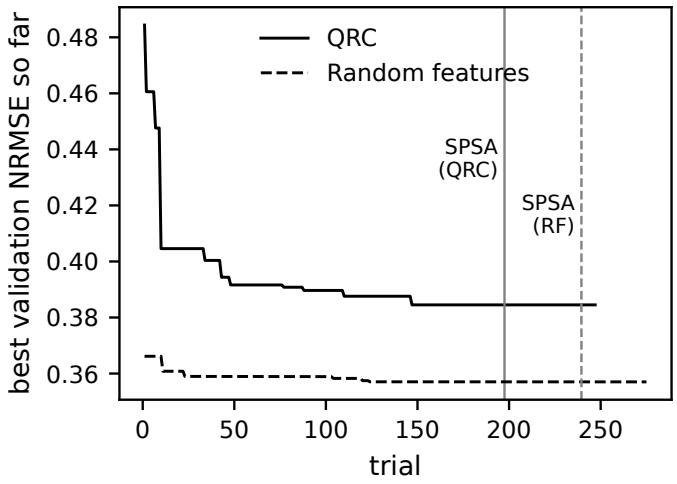


Figure 3: Best validation NRMSE so far against trial number, window, Ising, Z/ZZ+X/XX, for the QRC (solid) and the random features (dashed). The vertical lines mark where TPE stopped and SPSA started for each model; neither curve moves after them.

4.4 Free-running forecasts

A one-step score does not say how long a model stays useful when it is fed its own output. We therefore ran every frozen model of the eight cells autonomously. The readout is the one fit for the test score. From each of 20 starts spread evenly over the test slice the model predicts the next sample from the true history, and from then on each prediction is fed back as the next input. The window models rebuild their features from the last two inputs, the QRC drawing fresh shots at every step; the recurrent QRC and the ESN are first run over the true data up to the start and then continue from their own state, the recurrent QRC also resampling its shots every step. The valid prediction time (VPT) is the number of steps before the error, the RMS over the six variables of the error divided by each variable's test-slice s.d., first exceeds 0.5, over a horizon of 10 Lyapunov times (41 steps). It is averaged over the starts of a run and reported in Lyapunov times. The largest Lyapunov exponent of the task's own integrator (Benettin's method on the same RK4 step) is $\lambda = 0.973$ for $K = 6$, $F = 8$, so one Lyapunov time is 4.11 samples. As a check, the same routine gives 1.70 for $K = 40$, $F = 8$, close to the value of about 1.68 reported for that system [13]. Paired statistics are those of Section 3, applied to the VPT (Tables 7, 8 and 9).

No model forecasts far. No model's mean VPT reaches one Lyapunov time. The best is the ESN at width 156, 0.80 Lyapunov times, and the longest mean VPT of any single run is 1.08. One sample is 0.24 Lyapunov times, so these readouts, trained for one step at this coarse sampling, stay valid for at most a few steps. They are poor autonomous forecasters, and the free run compares them with each other, not with forecasters built for long horizons.

The ordering against the random features and the ESN holds. In every recurrent cell the ESN has the longest VPT, then the random features, then the QRC; the QRC wins no data seed against the ESN and at most 1 of 5 against the random features. With one basis the recurrent QRC's deficit against the random features is not significant (Holm p from 0.082 to 0.153); with two it is (p from 6.8×10^{-3} to 9.5×10^{-3}). In every window cell the random features outlast the QRC, decisively with one basis ($p \leq 3.5 \times 10^{-4}$, where the QRC's VPT of 0.08 to 0.13 Lyapunov times is close to the linear model's) and narrowly with two (p from 0.020 to 0.035). With one basis the QRC also loses its one-step lead over the simpler models: in the window cells it ties the linear model (XXZ: 12 of 25 runs, 2 of 5 seeds, Holm $p = 0.597$; Ising: 19 runs, 5 seeds, $p = 0.076$), and in the recurrent cells it ties Poly2 (XXZ: 10 runs, 1 seeds, $p = 0.937$; Ising: 12 runs, 3 seeds, $p = 0.796$).

Poly2 does not survive its own output. With the second basis the QRC outlasts Poly2 on all 25 runs and all 5 data seeds in all four cells, the one place in the grid where the QRC beats a classical model other than the linear one. The reason is Poly2, not the QRC: fed its own predictions, the quadratic map diverges, with a mean NRMSE of 8.3×10^4 (window) and 1.3×10^5 (recurrent) after one Lyapunov time. With one basis the QRC loses to Poly2 in the window cells ($p \leq 8.5 \times 10^{-4}$) and ties it in the recurrent cells (p from 0.796 to 0.937). The random-feature map, which also has bounded features, keeps its lead over the QRC.

Caveat on the horizon. One sample is 25 integrator steps of 0.01, so a Lyapunov time is only 4.11 samples, and the models are tuned for one step at that spacing. Ahmed et al. [2] sample at 0.01 time units and run their reservoirs in closed loop; they report VPTs of up to about 8.5 Lyapunov times on Lorenz-63 for both quantum and classical reservoirs. Our VPTs are much shorter and are not comparable with theirs; they compare the models of this grid with one another.

Table 7: Free-running forecasts: mean VPT in Lyapunov times ± s.d. over the 25 evaluation runs, with the mean VPT in samples in parentheses (4.11 samples per Lyapunov time; 20 starts per run, threshold 0.5). Bold: longest VPT in the cell.

Res.	Readout	QRC	RF	Poly2	Linear	ESN
window						
XXZ	Z/ZZ	0.08 ± 0.03 (0.3)	$\mathbf{0.31} \pm 0.05$ (1.3)	0.24 ± 0.03 (1.0)	0.07 ± 0.04 (0.3)	–
XXZ	+X/XX	0.37 ± 0.06 (1.5)	$\mathbf{0.45} \pm 0.10$ (1.9)	0.24 ± 0.03 (1.0)	0.07 ± 0.04 (0.3)	–
Ising	Z/ZZ	0.13 ± 0.04 (0.5)	$\mathbf{0.31} \pm 0.05$ (1.3)	0.24 ± 0.03 (1.0)	0.07 ± 0.04 (0.3)	–
Ising	+X/XX	0.41 ± 0.09 (1.7)	$\mathbf{0.45} \pm 0.10$ (1.9)	0.24 ± 0.03 (1.0)	0.07 ± 0.04 (0.3)	–
recurrent						
XXZ	Z/ZZ	0.22 ± 0.05 (0.9)	0.29 ± 0.06 (1.2)	0.22 ± 0.02 (0.9)	0.08 ± 0.02 (0.3)	$\mathbf{0.62} \pm 0.10$ (2.5)
XXZ	+X/XX	0.34 ± 0.08 (1.4)	0.48 ± 0.07 (2.0)	0.22 ± 0.02 (0.9)	0.08 ± 0.02 (0.3)	$\mathbf{0.80} \pm 0.10$ (3.3)
Ising	Z/ZZ	0.22 ± 0.06 (0.9)	0.29 ± 0.06 (1.2)	0.22 ± 0.02 (0.9)	0.08 ± 0.02 (0.3)	$\mathbf{0.62} \pm 0.10$ (2.5)
Ising	+X/XX	0.38 ± 0.07 (1.6)	0.48 ± 0.07 (2.0)	0.22 ± 0.02 (0.9)	0.08 ± 0.02 (0.3)	$\mathbf{0.80} \pm 0.10$ (3.3)

Table 8: QRC wins on VPT against each classical model, per cell: runs (of 25) and, in parentheses, data seeds (of 5) on which the QRC's VPT is the longer. † as in Table 2.

Kind	Res.	Readout	vs Random features	vs Poly2	vs Linear	vs ESN
window	XXZ	Z/ZZ	0/25 (0/5)	0/25 (0/5)†	12/25 (2/5)†	–
window	XXZ	+X/XX	1/25 (0/5)	25/25 (5/5)†	25/25 (5/5)†	–
window	Ising	Z/ZZ	0/25 (0/5)	0/25 (0/5)†	19/25 (5/5)†	–
window	Ising	+X/XX	8/25 (1/5)	25/25 (5/5)†	25/25 (5/5)†	–
recurrent	XXZ	Z/ZZ	2/25 (0/5)	10/25 (1/5)†	25/25 (5/5)†	0/25 (0/5)
recurrent	XXZ	+X/XX	2/25 (0/5)	25/25 (5/5)†	25/25 (5/5)†	0/25 (0/5)
recurrent	Ising	Z/ZZ	4/25 (1/5)	12/25 (3/5)†	25/25 (5/5)†	0/25 (0/5)
recurrent	Ising	+X/XX	2/25 (0/5)	25/25 (5/5)†	25/25 (5/5)†	0/25 (0/5)

Table 9: Holm-corrected p of the QRC against each classical model on VPT, per cell, as in Table 3.

Kind	Res.	Readout	vs Random features	vs Poly2	vs Linear	vs ESN
window	XXZ	Z/ZZ	3.5×10^{-4}	3.5×10^{-4}	0.597	–
window	XXZ	+X/XX	0.020	1.0×10^{-3}	1.0×10^{-3}	–
window	Ising	Z/ZZ	2.8×10^{-4}	8.5×10^{-4}	0.076	–
window	Ising	+X/XX	0.035	1.9×10^{-3}	4.1×10^{-4}	–
recurrent	XXZ	Z/ZZ	0.082	0.937	9.6×10^{-4}	4.8×10^{-4}
recurrent	XXZ	+X/XX	9.5×10^{-3}	9.5×10^{-3}	3.5×10^{-3}	4.2×10^{-4}
recurrent	Ising	Z/ZZ	0.153	0.796	1.6×10^{-3}	1.6×10^{-3}
recurrent	Ising	+X/XX	6.8×10^{-3}	3.6×10^{-3}	1.4×10^{-3}	9.1×10^{-4}

5 Discussion

5.1 What would it take for the QRC to win here?

More bases, not more shots per basis. Halving the shots per basis to add the X basis lowered the QRC's error in every cell. In the window cells, where the gain is large, this suggests that at 12288 shots per step the binding limit is what is measured, not how precisely. The X-basis gain is, however, confounded with the doubled width: we did not run a one-basis QRC of width 156, so how much of the gain is the new basis and how much is more features is not separated. A third basis is the obvious next test. The comparison has to stay width-matched, though: the random features gained 0.078 to 0.084 from the doubled width alone, so a new basis must add more than width does.

Ranges at their edges. In the two-basis cells the tuned reservoir parameters sit at the top of their ranges: $v = 9.65$ (window, Ising) and $v = 9.66$ (recurrent, Ising) with $v \leq 10$, and $W = 9.95$ (recurrent, XXZ) with $W \leq 10$. The QRC optimum may lie outside the searched range. The same holds for the ESN, whose spectral radius sits at 0.052 or below against a lower bound of 0.05 and whose bias scale sits at 9.80 or above against an upper bound of 10 in every recurrent cell. The ridge penalty sits at its lower bound 10^{-6} (at most 1.28×10^{-6}) for the random features in all four one-basis cells and for the QRC in the window, XXZ, two-basis cell. Widening the ranges is cheap and should be done for both sides together.

The hardware cost of the restarting protocol. The restarting protocol keeps the simulated memory exact, but on hardware each measured step is a separate preparation. For the reset QRC each step circuit has two drives, so the cost is 12288 shots of a short circuit per step. For the recurrent QRC the circuit of step t replays all t drives before it, so one series of 3000

steps needs $3000(3000+1)/2 \approx 4.5 \times 10^6$ drives per shot round, each repeated 12288 times, and the feedback angles must be fixed from the counts of earlier steps. The recurrent QRC is thus the more expensive of the two memory schemes and, with two bases, also the weaker. Weak-measurement and rewinding protocols [9] reduce the replay cost; whether their extra noise leaves the QRC competitive is open.

One step against many. The models are tuned for one step, and at our sampling none of them stays valid in free run for one Lyapunov time (Section 4.4). The free run keeps the one-step ordering against the random features and the ESN, so it gives the QRC no hidden advantage. A fair test of long-horizon forecasting would sample finely, as Ahmed et al. [2] do, and tune every model on the free-running metric itself.

5.2 Why earlier claims differ

The six patterns of Section 2 each predict a direction in our grid. *Resource units* (pattern 1): matching features rather than qubits against neurons removes the readout-size difference; at equal width the random features win. *Baseline tuning* (pattern 2): our own records show how much one knob matters. In the first exact-expectation run of the recurrent XXZ cell the ESN had no bias term; it reached 0.445 against the QRC's 0.412, and the QRC won 23 of 25 runs. Adding the bias scale to the ESN's search, with the QRC unchanged, gives 0.340, and the ESN wins every run (Section 7 lists both records). *QRC against QRC* (pattern 3): the second basis is a real gain over the one-basis QRC, and reporting it against that baseline alone would read as a success; against the width-matched random features it is not. *Capacity metrics* (pattern 4): we report forecast error only, and the gap to a classical model is what decides the comparison. *Exact expectations* (pattern 5): Section 4.2 shows a tie under exact values becoming a loss on every seed under shots. *Few seeds* (pattern 6): the s.d. over runs is 0.009 to 0.018 for the QRC (Table 1), so single-run gaps smaller than that are not interpretable; the margins we report are larger than the run-to-run spread and hold on every data seed.

6 Limitations

The study has one task (Lorenz-96 with $K = 6$ and $F = 8$), one system size ($q = 12$), one input window (two steps), two reservoir families with all-to-all couplings, and one tuning horizon (one step). The free-running test uses coarse sampling (one sample per 25 integrator steps, 4.11 samples per Lyapunov time), so its VPTs resolve only a few steps, and it uses a single error threshold (0.5); models tuned for free running at finer sampling might rank differently. The searched ranges are finite, and several optima sit at their edges on both the quantum and the classical side. Shots are simulated by sampling from the exact state; there is no gate, readout or decoherence noise, so hardware results would be worse for the QRC, not better. The feedback-noise explanation of Section 4.2 is not tested by an ablation. All shot results use one budget (12288 per step); the shot-noise contrast rests on one XXZ, single-basis pair per memory scheme, and the refinement stage was tested in one cell. The Holm correction is applied within each cell, with no correction across the 28 tests of the grid. For scale, the no-change forecast scores NRMSE 1.13 to 1.16 on the test slices: at this sampling the last value is a worse forecast than the mean, so one step ahead is already a long way on this attractor. The exact-expectation reference for the window cell used a narrower random-feature range than the shot runs. The results say nothing about larger reservoirs, other encodings, quantum input data or other tasks.

7 Reproducibility

All runs use the `qrc_bench` pipeline (https://github.com/pandey-tushar/Quantum_Chaos_solver). Each record stores its protocol, the full trial history of every model, the per-run test scores, package versions, the GPU and the git commit of the pipeline (Table 10). The runs used one NVIDIA GeForce RTX 2050 GPU with 4 GB of memory; the records listed take 22.1 hours of wall time in total, almost all of it in the QRC's tuning. Two records need a note. The record `l96_recurrent_xxz_v2` reloaded the completed QRC study of `l96_recurrent_xxz` (its QRC tuning took 0.14 s; the original took 3.31 h) and re-tuned the classical models with the ESN bias knob and the wider random-feature range; it carries the same commit as the earlier record, so these changes were not yet committed when it ran. The refinement record carries commit `bccf36d`; its SPSA stage ran from changes on top of that commit, later committed as `df03eeb`. The free-run records (`freerun_<kind>.json` next to each cell's record) carry commit `df03eeb`; the free-run code ran from changes not yet committed on top of it, and the eight free runs took 0.66 h in total. This paper's tables and inline numbers are generated by `make_tables.py` and its figures by `make_figures.py`, both of which read the records directly.

Table 10: Records, pipeline commit (`environment.git_commit`), total wall time (`total_seconds`) and the QRC's tuning time.

Record	commit	total (h)	QRC tuning (h)
`l96_window_xxz_shots`	`a902b04`	0.39	0.35
`l96_window_xxz_zx_shots`	`bccf36d`	0.51	0.43
`l96_window_ising_shots`	`a902b04`	0.64	0.59
`l96_window_ising_zx_shots`	`bccf36d`	1.11	1.03
`l96_recurrent_xxz_shots`	`a902b04`	1.64	1.59
`l96_recurrent_xxz_zx_shots`	`bc6e881`	2.92	2.79
`l96_recurrent_ising_shots`	`bc6e881`	2.14	2.09
`l96_recurrent_ising_zx_shots`	`bccf36d`	2.72	2.57
`l96_window_xxz`	`4538057`	4.84	4.35
`l96_recurrent_xxz`	`4538057`	3.51	3.31
`l96_recurrent_xxz_v2`	`4538057`	0.29	0.00
`l96_window_ising_zx_refine`	`bccf36d`	1.39	1.29

Acknowledgment

The author used Claude (Anthropic) to assist with running and optimizing the simulation code and with drafting. All technical content, results, and conclusions are the author's own.

References

[1] Keisuke Fujii and Kohei Nakajima. Harnessing disordered-ensemble quantum dynamics for machine learning. *Physical Review Applied*, 8:024030, 2017. arXiv:1602.08159.

[2] Osama Ahmed, Felix Tennie, and Luca Magri. Prediction of chaotic dynamics and extreme events: A recurrence-free quantum reservoir computing approach, 2024. arXiv:2405.03390.

[3] Kaito Kobayashi, Keisuke Fujii, and Naoki Yamamoto. Feedback-driven quantum reservoir computing for time-series analysis. *PRX Quantum*, 5:040325, 2024. arXiv:2406.15783.

[4] Phi Hung Nguyen and Daniel Soh. Quantum-fisher-information-aware optimization yields a quantum reservoir advantage over tuned classical reservoirs, 2026. arXiv:2610.14454.

[5] Tushar Pandey. When classical baselines are tuned as carefully as the quantum model, does quantum reservoir computing still win?, 2026. arXiv:2607.09905.

[6] Tushar Pandey. A quantum reservoir architecture for chaotic forecasting and a test of whether its high dimension helps, 2026. arXiv:2607.07978.

[7] Tushar Pandey. An architectural capacity ceiling, not a barren plateau: why a fixed-encoding variational quantum circuit cannot fit the Lorenz-63 attractor, 2026. arXiv:2604.23743.

[8] Rodrigo Martínez-Pe na, Gian Luca Giorgi, Johannes Nokkala, Miguel C. Soriano, and Roberta Zambrini. Dynamical phase transitions in quantum reservoir computing. *Physical Review Letters*, 127:100502, 2021. arXiv:2103.05348.

[9] Pere Mujal, Rodrigo Martínez-Pe na, Gian Luca Giorgi, Miguel C. Soriano, and Roberta Zambrini. Time series quantum reservoir computing with weak and projective measurements. *npj Quantum Information*, 9:16, 2023. arXiv:2205.06809.

[10] Fangjun Hu, Gerasimos Angelatos, Saeed A. Khan, Marti Vives, Esin Türeci, Leon Bello, Graham E. Rowlands, Guilhem J. Ribeill, and Hakan E. Türeci. Quantifying the expressive capacity of quantum systems: Fundamental limits and eigentasks. *Physical Review X*, 13:041020, 2023. arXiv:2301.00042.

[11] Wissal Hamhoum, Soumaya Cherkaoui, Jean-Frederic Laprade, Ola Ahmad, and Shengrui Wang. Multivariate time series forecasting with gate-based quantum reservoir computing on NISQ hardware, 2025. arXiv:2510.13634.

[12] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework, 2019. arXiv:1907.10902.

[13] P. R. Vlachas, J. Pathak, B. R. Hunt, T. P. Sapsis, M. Girvan, E. Ott, and P. Koumoutsakos. Backpropagation algorithms and reservoir computing in recurrent neural networks for the forecasting of complex spatiotemporal dynamics, 2020. arXiv:1910.05266.